

Coin Change (Count Ways)

```
import java.util.*;

public class CoinChangeWays {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt(); // number of coins
        int amount = sc.nextInt(); // target amount
        int[] coins = new int[n];
        for (int i = 0; i < n; i++) coins[i] = sc.nextInt();

        int[] dp = new int[amount + 1];
        dp[0] = 1; // one way to make amount 0 (use no coins)

        for (int coin : coins) {
            for (int i = coin; i <= amount; i++) {
                dp[i] += dp[i - coin];
            }
        }

        System.out.println(dp[amount]);
        sc.close();
    }
}
```

Sample Input 1

```
3 4
1 2 3
```

Output

```
4
```

Dijkstra's Algorithm

```
import java.util.*;

public class Main {
    static class Edge {
        int to, weight;
        Edge(int t, int w) {
            to = t;
        }
    }
}
```

```

        weight = w;
    }
}

static void addEdge(List<List<Edge>> graph, int u, int v, int w) {
    graph.get(u).add(new Edge(v, w)); // add  $u \rightarrow v$ 
    graph.get(v).add(new Edge(u, w)); // since undirected, add  $v \rightarrow u$ 
}

static void dijkstra(List<List<Edge>> graph, int V, int src) {
    int[] dist = new int[V];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[src] = 0;

    PriorityQueue<int[]> pq = new PriorityQueue<>(Comparator.comparingInt(a -> a[1]));
    pq.add(new int[] {src, 0});

    while (!pq.isEmpty()) {
        int[] cur = pq.poll();
        int node = cur[0];
        int d = cur[1];

        if (d > dist[node]) continue;

        for (Edge e : graph.get(node)) {
            if (dist[node] + e.weight < dist[e.to]) {
                dist[e.to] = dist[node] + e.weight;
                pq.add(new int[] {e.to, dist[e.to]});
            }
        }
    }
}

System.out.print("Shortest distances from vertex " + src + " to all other vertices: ");
for (int i = 0; i < V; i++) {
    System.out.print(dist[i] + " ");
}
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int V = sc.nextInt();
    int E = sc.nextInt();

    List<List<Edge>> graph = new ArrayList<>();

```

```

for (int i = 0; i < V; i++) graph.add(new ArrayList<>());

for (int i = 0; i < E; i++) {
    int from = sc.nextInt();
    int to = sc.nextInt();
    int weight = sc.nextInt();
    addEdge(graph, from, to, weight);
}

int S = sc.nextInt();
dijkstra(graph, V, S);
}
}

```

Input

```

3
3
0 1 1
0 2 6
1 2 3
2

```

Output

Shortest distances from vertex 2 to all other vertices: 4 3 0

HUFFMAN CODING

```

import java.util.*;

class Node {
    char ch;
    int freq;
    Node left, right;

    Node(char ch, int freq) {
        this.ch = ch;
        this.freq = freq;
    }

    Node(int freq, Node left, Node right) {
        this.ch = '-'; // internal node
        this.freq = freq;
    }
}

```

```

        this.left = left;
        this.right = right;
    }
}

```

```

public class Main {

```

```

    // Recursive function to print Huffman codes
    static void printCodes(Node root, String code) {
        if (root == null) return;

```

```

        // if it's a leaf node, print the character and its code
        if (root.left == null && root.right == null) {
            System.out.println(root.ch + ": " + code);
            return;
        }

```

```

        printCodes(root.left, code + "0");
        printCodes(root.right, code + "1");
    }

```

```

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

```

```

        PriorityQueue<Node> pq = new PriorityQueue<>(Comparator.comparingInt(a ->
a.freq));

```

```

        // Input characters and frequencies
        for (int i = 0; i < n; i++) {
            char ch = sc.next().charAt(0);
            int freq = sc.nextInt();
            pq.add(new Node(ch, freq));
        }

```

```

        // Build Huffman Tree
        while (pq.size() > 1) {
            Node left = pq.poll();
            Node right = pq.poll();
            Node parent = new Node(left.freq + right.freq, left, right);
            pq.add(parent);
        }

```

```

        // Root of Huffman Tree

```

```

        Node root = pq.poll();

        // Print Huffman Codes
        printCodes(root, "");
    }
}

```

Input

```

4
A 45
B 13
C 12
D 16

```

Output

```

D: 00
C: 010
B: 011
A: 1

```

FRACTIONAL KNAPSACK

```

import java.util.*;

class Item {
    int weight, value;
    double ratio;

    Item(int w, int v) {
        weight = w;
        value = v;
        ratio = (double) v / w;
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt(); // number of items
        int[] weights = new int[n];
        int[] values = new int[n];
    }
}

```

```

for (int i = 0; i < n; i++)
    weights[i] = sc.nextInt();
for (int i = 0; i < n; i++)
    values[i] = sc.nextInt();
int capacity = sc.nextInt();

// Create list of items
Item[] items = new Item[n];
for (int i = 0; i < n; i++) items[i] = new Item(weights[i], values[i]);

// Sort items by value/weight ratio in descending order
Arrays.sort(items, (a, b) -> Double.compare(b.ratio, a.ratio));

double totalValue = 0.0;
int remaining = capacity;

for (Item item : items) {
    if (remaining == 0) break;

    if (item.weight <= remaining) {
        totalValue += item.value;
        remaining -= item.weight;
    } else {
        totalValue += item.ratio * remaining;
        remaining = 0;
    }
}

System.out.printf("Objects worth Rs.%.2f\n", totalValue);
sc.close();
}
}

```

Input 1

```

3
10 20 30
60 100 120
50

```

Output

Objects worth Rs.240.00

Job Sequencing Problem

```
import java.util.*;

class Job {
    int deadline, profit;
    Job(int d, int p) {
        deadline = d;
        profit = p;
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt(); // number of jobs
        Job[] jobs = new Job[n];

        for (int i = 0; i < n; i++) {
            int d = sc.nextInt();
            int p = sc.nextInt();
            jobs[i] = new Job(d, p);
        }

        // Sort jobs by profit (descending order)
        Arrays.sort(jobs, (a, b) -> b.profit - a.profit);

        // Find max deadline
        int maxDeadline = 0;
        for (Job job : jobs) maxDeadline = Math.max(maxDeadline, job.deadline);

        // Time slots to schedule jobs
        boolean[] slot = new boolean[maxDeadline + 1];

        int totalProfit = 0;

        // Try to schedule each job
        for (Job job : jobs) {
            for (int j = job.deadline; j > 0; j--) {
                if (!slot[j]) { // if slot is free
                    slot[j] = true;
                    totalProfit += job.profit;
                    break;
                }
            }
        }
    }
}
```

```

    }
    }
}

System.out.println("Total Profit = " + totalProfit);
sc.close();
}
}

```

Input 1

```

4
4 80
3 40
2 30
1 60

```

Output

Total Profit = 210

INTERVAL SCHEDULING

```

import java.util.*;
class Main{
    static class Job{
        String id;
        int s;
        int f;
        Job(String id,int s,int f){
            this.id=id;
            this.s=s;
            this.f=f;
        }
    }
    public static void main(String[]args){
        Scanner sc=new Scanner(System.in);
        int n=sc.nextInt();
        sc.nextLine();
        Job[]jobs=new Job[n];

        for(int i=0;i<n;i++){
            String[] line=sc.nextLine().split(" ");
            String id=line[0];
            int s=Integer.parseInt(line[1]);

```



```

        int f=Integer.parseInt(line[2]);
        jobs[i]=new Job(id,s,f);
    }

    Arrays.sort(jobs,(a,b)->(a.f-b.f));
    List<Job>list=new ArrayList<>();
    int lf=-1;
    int count=0;
    for(Job j:jobs){
        if(j.s>=lf){
            list.add(j);
            lf=j.f;
            count++;
        }
    }
    for(Job job:list){
        System.out.println(job.id+" (" +job.s+", "+job.f+"");
    }
    System.out.println(count);
}
}

```

Input 1 :

```

3
a 0 2
b 2 4
c 4 6

```

Output 1 :

```

a (0, 2)
b (2, 4)
c (4, 6)
3

```

WEIGHTED INTERVAL SCHEDULING

```

import java.util.*;
class Main{
    static class Interval{
        int s;
        int e;
        int w;
        Interval(int s,int e,int w){
            this.s=s;

```

```

        this.e=e;
        this.w=w;
    }
}
public static void main(String[] args){
    Scanner sc=new Scanner(System.in);
    int n=sc.nextInt();
    Interval[] is=new Interval[n];
    for(int i=0;i<n;i++){
        is[i]=new Interval(sc.nextInt(),sc.nextInt(),sc.nextInt());
    }
    int r=Find(is);
    System.out.println(r);
}
public static int Find(Interval[] is){
    int n=is.length;
    int[] dp=new int[n];
    Arrays.sort(is,Comparator.comparingInt(i->i.e));
    dp[0]=is[0].w;
    for(int i=1;i<n;i++){
        int incl=is[i].w;
        for(int j=i-1;j>=0;j--){
            if(is[j].e<=is[i].s){
                incl+=dp[j];
                break;
            }
        }
        dp[i]=Math.max(incl,dp[i-1]);
    }
    return dp[n-1];
}
}

```

Input 1 :

```

4
1 3 10
3 5 20
2 6 25
5 7 15

```

Output 1 :

```

45

```

MULTISTAGE GRAPH

```
import java.util.*;
class Main{
    static final int INF=9999;
    public static void main(String[]args){
        Scanner sc=new Scanner(System.in);
        int n=sc.nextInt();
        int[][]cost=new int[n][n];
        for(int i=0;i<n;i++){
            for(int j=0;j<n;j++){
                cost[i][j]=sc.nextInt();
                if(i!=j && cost[i][j]==0){
                    cost[i][j]=INF;
                }
            }
        }
        System.out.println("Shortest distance from hub 0 to hub "+(n-1)+" is:
"+Distance(cost,n));
    }
    public static int Distance(int[][]cost,int n){
        int[]dp=new int[n];
        Arrays.fill(dp,INF);
        dp[n-1]=0;
        for(int i=n-2;i>=0;i--){
            for(int j=i+1;j<n;j++){
                if(cost[i][j]!=INF && cost[i][j]+dp[j]<dp[i]){
                    dp[i]=cost[i][j]+dp[j];
                }
            }
        }
        return dp[0];
    }
}
```

Input 1 :

```
4
0 10 0 0
0 0 20 0
0 0 0 30
0 0 0 0
```

Output 1 :

Shortest distance from hub 0 to hub 3 is: 60

SUBSET SUM PROBLEM

```
import java.util.*;
class Main{
    public static List<List<Integer>>res=new ArrayList<>();
    public static void main(String[]args){
        Scanner sc=new Scanner(System.in);
        int n=sc.nextInt();
        int[]nums=new int[n];
        for(int i=0;i<n;i++){
            nums[i]=sc.nextInt();
        }
        int target=sc.nextInt();
        int cb=(target+10)*3;
        System.out.println(cb);
        backtrack(nums,0,target,new ArrayList<>());
        if(res.size()==0){
            System.out.println("No combination of products");
        }
        else{
            for(List<Integer>val:res){
                for(int v:val){
                    System.out.print(v+" ");
                }
            }
        }
    }
    public static void backtrack(int[]nums,int s,int t,List<Integer>path){
        int sum=0;
        for(int num:path){
            sum+=num;
        }
        if(sum==t){
            res.add(new ArrayList<>(path));
        }
        for(int i=s;i<nums.length;i++){
            path.add(nums[i]);
            backtrack(nums,i+1,t,path);
            path.remove(path.size()-1);
        }
    }
}
```

Input 1 :

4
21 28 31 37
80

Output 1 :

270
21 28 31

GRAPH COLORING

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int v = sc.nextInt(); // number of vertices
        int[][] graph = new int[v][v];
        for (int i = 0; i < v; i++) {
            for (int j = 0; j < v; j++) {
                graph[i][j] = sc.nextInt();
            }
        }

        int[] color = new int[v];
        Arrays.fill(color, -1);
        color[0] = 0;
        boolean[] used = new boolean[v];
        for (int u = 1; u < v; u++) {
            Arrays.fill(used, false);
            for (int i = 0; i < v; i++) {
                if (graph[u][i] == 1 && color[i] != -1)
                    used[color[i]] = true;
            }
            int c;
            for (c = 0; c < v; c++) {
                if (!used[c])
                    break;
            }
            color[u] = c; // assign the found color
        }
        int maxColor = 0;
```

```

        for (int c : color) maxColor = Math.max(maxColor, c);

        System.out.println("Chromatic Number of the graph is: " + (maxColor + 1));
    }
}

```

Input 1 :

```

4
0 1 1 1
1 0 1 0
1 1 0 1
1 0 1 0

```

Output 1 :

Chromatic Number of the graph is: 3

LCS

```

import java.util.*;

class Main {
    public static void main(String[] args) {
        String s1 = "BCADB";
        String s2 = "BAM";
        int m = s1.length(), n = s2.length();
        int[][] dp = new int[m + 1][n + 1];

        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (s1.charAt(i - 1) == s2.charAt(j - 1))
                    dp[i][j] = 1 + dp[i - 1][j - 1];
                else
                    dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
            }
        }

        StringBuilder lcs = new StringBuilder();
        int i = m, j = n;
        while (i > 0 && j > 0) {
            if (s1.charAt(i - 1) == s2.charAt(j - 1)) {
                lcs.append(s1.charAt(i - 1));
                i--; j--;
            } else if (dp[i - 1][j] > dp[i][j - 1])
                i--;
        }
    }
}

```

```
        else
            j--;
    }

    System.out.println(lcs.reverse().toString());
}
}
```

Input

ABCB
DAB
BDCAB

Output

BDAB